

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Дисциплина:

“Программирование”

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

Объектно-ориентированное программирование в Python

Вариант: 3-5

Выполнила:

Студентка гр. 3148 Жук Анастасия



Проверил:

Волков Александр Григорьевич

Санкт-Петербург
2024г.

Задание

Разработать на языке Python для ОС Linux модуль, содержащий определение заданного типа, и тестовый модуль для pytest, содержащий набор тестов для проверки корректности определения типа.

Базовый класс: deque (из модуля collections).

Методы класса:

Очередь (deque)

Метод	Описание	Объем вычислений
append()	Добавление нового элемента справа (в конец списка)	$O(1)$
appendleft()	Добавление нового элемента слева (в начало списка)	$O(1)$
pop()	Удаление элемента справа (с конца списка)	$O(1)$
popleft()	Удаление элемента слева (с начала списка)	$O(1)$
extend()	Добавление нескольких m элементов справа (в конец списка)	$O(m)$
extendleft()	Добавление нескольких m элементов слева (в начало списка)	$O(m)$
insert()	Вставка нового элемента в произвольную позицию	$O(n)$
remove()	Удаление элемента списка с указанным значением (удаляется первый найденный элемент с заданным значением)	$O(n)$
clear()	Удаление всех элементов списка	$O(1)$
copy()	Создание копии двусвязного списка	$O(n)$

Формат данных: адрес электронной почты.

Программа

lab6avzN3148.py

```
from collections import deque
import re

class FormatError(Exception): pass

class UndoError(Exception): pass

class RedoError(Exception): pass

class MyDeque(deque):
    @staticmethod
    def _email(email): #проверка на адрес электронной почты
        pattern = re.compile(r"^[a-zA-Z0-9_+]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+$")
        return bool(pattern.match(email))

    def Type_check(self, value):
        if not isinstance(value, str):
            raise TypeError("Ошибка: добавляемый элемент не строка!")
        if not self._email(value):
            raise FormatError("Ошибка: добавляемая строка не адрес электронной почты!")
        return None

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        for i in self:
            self.Type_check(i)
        self._history = []
        self._redo_history = []

    def append(self, item):
        #для корректности работы программы при использовании undo() и redo()
        if self != self._history and self != self._redo_history:
            self.Type_check(item)
            self._history.append(deque(self))
            self._redo_history.clear()
            return super().append(item)

    def appendleft(self, item):
        self.Type_check(item)
        self._history.append(deque(self))
        self._redo_history.clear()
        return super().appendleft(item)
```

```

def pop(self):
    #если нечего удалять, то пропускаем этот шаг (применение pop) без исключений
    if not self:
        return None
    self._history.append(deque(self))
    self._redo_history.clear()
    return super().pop()

def popleft(self):
    #если нечего удалять, то пропускаем этот шаг (применение popleft) без исключений
    if not self:
        return None
    self._history.append(deque(self))
    self._redo_history.clear()
    return super().popleft()

def extend(self, item):
    #проверка добавляемых элементов
    for i in item:
        self.Type_check(i)
    self._history.append(deque(self))
    self._redo_history.clear()
    return super().extend(item)

def extendleft(self, item):
    # проверка добавляемых элементов
    for i in item:
        self.Type_check(i)
    self._history.append(deque(self))
    self._redo_history.clear()
    return super().extend(item)

def insert(self, index, item):
    self.Type_check(item)
    print(len(self))
    #индекс должен быть числом
    if isinstance(index, int):
        self._history.append(deque(self))
        self._redo_history.clear()
        return super().insert(index, item)

def remove(self, item):
    self.Type_check(item)
    if item in self:
        self._history.append(deque(self))
        self._redo_history.clear()

```

```

        return super().remove(item)

def clear(self):
    self._history.append(deque(self))
    self._redo_history.clear()
    return super().clear()

def undo(self):
    if not self._history:
        raise UndoError("Ошибка: нет шагов для undo!")
    self._redo_history.append(deque(self)) #фиксируем текущее состояние
    super().clear()
    super().extend(self._history[-1]) #делаем шаг назад
    self._history = self._history[:-1]

def redo(self):
    if not self._redo_history:
        raise RedoError("Ошибка: нет шагов для redo!")
    self._history.append(deque(self)) #фиксируем текущее состояние
    super().clear()
    super().extend(self._redo_history[-1]) #возврат
    self._redo_history = self._redo_history[:-1]

```

test-lab6avzN3148.py

```

from lab6avzN3148 import MyDeque, FormatError, UndoError, RedoError

```

```

def test_1():
    #Различные варианты добавления в очередь адресов
    MD = MyDeque(['kate@yandex.ru'])
    MD.append('nastenkazhuk05@mail.ru')
    MD.insert(1, 'dom@gmail.com')
    MD.appendleft('book2005@mail.ru')
    assert MD == MyDeque(['book2005@mail.ru', 'kate@yandex.ru', 'dom@gmail.com',
'nastenkazhuk05@mail.ru'])

```

```

def test_2():
    #Неверный ввод адреса - неверный формат
    MD = MyDeque([])
    try:
        MD.append('dfkfgghiughirt')
    except FormatError:
        err = 1
    assert err == 1

```

```
def test_3():
    # Неверный ввод адреса - неправильный тип
    MD = MyDeque([])
    try:
        MD.append(521)
    except TypeError:
        err = 1
    assert err == 1
```

```
def test_4():
    #Всевозможное удаление элементов
    MD = MyDeque(['book2005@mail.ru', 'nastenkazhuk05@mail.ru', 'kate@yandex.ru',
'nastenkazhuk05@mail.ru', 'dom@gmail.com'])
    MD.pop()
    MD.popleft()
    MD.remove('nastenkazhuk05@mail.ru')
    assert MD == MyDeque(['kate@yandex.ru', 'nastenkazhuk05@mail.ru'])
```

```
def test_5():
    #Проверим в действии функции, которые добавляют сразу несколько элементов
    MD = MyDeque([])
    MD.extend(['book2005@mail.ru', 'nastenkazhuk05@mail.ru'])
    MD.extendleft(['kate@yandex.ru', 'dom@gmail.com'])
    assert MD == MyDeque(['book2005@mail.ru', 'nastenkazhuk05@mail.ru', 'kate@yandex.ru',
'dom@gmail.com'])
```

```
def test_6():
    # Проверим в действии функции, которые добавляют сразу несколько элементов, среди
    # которых есть не адрес
    MD = MyDeque([])
    try:
        MD.extend(['book2005@mail.ru', 'nastenkazhuk05mail.ru'])
    except:
        err = 1
    assert err == 1
```

```
def test_7():
    #Проверим очистку очереди
    MD = MyDeque(['book2005@mail.ru', 'nastenkazhuk05@mail.ru', 'kate@yandex.ru',
'nastenkazhuk05@mail.ru', 'dom@gmail.com'])
    MD.clear()
    assert MD == MyDeque([])
```

```
def test_8():
    #Проверка работы методов undo() и redo() - хороший случай, без исключений
    MD = MyDeque(['nastenkazhuk05@mail.ru'])
    try:
        MD.append('kate@yandex.ru')
        MD.append('dom@gmail.com')
        MD.undo()
        MD.undo()
        MD.redo()
        MD.redo()
    except:
        err = 1
    assert MD == MyDeque(['nastenkazhuk05@mail.ru', 'kate@yandex.ru', 'dom@gmail.com'])
```

```
def test_9():
    #Проверка работы методов undo() и redo() - ошибка при запуске undo()
    MD = MyDeque(['nastenkazhuk05@mail.ru'])
    try:
        MD.append('kate@yandex.ru')
        MD.append('dom@gmail.com')
        MD.undo()
        MD.undo()
        MD.undo()
    except UndoError:
        err = 1
    assert err == 1
```

```
def test_10():
    #Проверка работы методов undo() и redo() - ошибка при запуске redo()
    MD = MyDeque(['nastenkazhuk05@mail.ru'])
    try:
        MD.append('kate@yandex.ru')
        MD.append('dom@gmail.com')
        MD.redo()
    except RedoError:
        err = 1
    assert err == 1
```

```
def test():
    MD = MyDeque(['dom@gmail.com'])
    MD.insert(100, 'sdf@d.h')
```

```
assert MD == MyDeque(['dom@gmail.com', 'sdf@d.h'])
```